

When Does Repairing Preference-Graph Inconsistency Improve Retrieval Quality? A Multi-Benchmark Data Quality Study

ANONYMOUS AUTHOR(S)

Retrieval systems that combine multiple ranking signals often represent disagreement between them as a directed preference graph, where an edge records evidence that one document should rank above another. When signals disagree enough, this graph can contain cycles, so no single ordering can satisfy every recorded preference — a structural data-quality defect in a derived artifact. A standard remedy repairs the graph by removing a minimum-weight set of cycle-inducing edges before extracting a final ranking. This paper asks whether that structural repair also improves the quality of the resulting ranking, measured by a standard graded-relevance retrieval metric. Across four public retrieval benchmarks and three vote-construction regimes, we find that vote construction, not repair, is the dominant driver of graph inconsistency, and that repair reliably improves graph-level consistency whenever inconsistency is present. This structural improvement, however, translates into a measurable ranking-quality gain in only one of twelve dataset-and-regime combinations tested, and even there the lower endpoint of the reported 95% interval is exactly 0.0. Simple score-fusion baselines that never inspect graph structure remain competitive with, or better than, every graph-repair method evaluated. A rule-based failure-class analysis identifies why repair is usually inactive, often invisible at the evaluation cutoff, and occasionally harmful, offering practical guidance on when this intervention is worth applying. We therefore treat structural consistency and retrieval effectiveness as distinct quality dimensions that must be evaluated separately rather than assuming that improvement in one guarantees improvement in the other.

CCS Concepts: • **Information systems** → **Data cleaning**; **Data quality**; **Retrieval models and ranking**.

Additional Key Words and Phrases: preference graphs, data quality, structural inconsistency, rank aggregation, feedback arc set repair, retrieval evaluation, benchmark curation

ACM Reference Format:

Anonymous Author(s). 2026. When Does Repairing Preference-Graph Inconsistency Improve Retrieval Quality? A Multi-Benchmark Data Quality Study. 1, 1 (July 2026), 29 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 INTRODUCTION

Modern retrieval pipelines rarely rely on a single ranking signal. It is increasingly common to combine sparse retrievers, dense retrievers, cross-encoders, and large language models operating in pointwise, pairwise, or listwise comparison modes, and to treat their outputs as evidence to be aggregated rather than as a final answer [2, 10, 14, 16]. A natural way to combine such heterogeneous signals is to represent them as a weighted, query-specific preference graph: a directed edge from document u to document v records evidence that u should be ranked ahead of v , and edge weight records the strength of that evidence [1, 5, 13]. This representation is attractive precisely because it keeps disagreement visible instead of collapsing it immediately into a single scalar score. But that same property is also its central liability from a data-quality standpoint: once preferences from multiple rankers are aggregated into a graph, the graph can contain directed cycles, meaning the aggregated evidence cannot be represented by any single, globally consistent ordering.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Association for Computing Machinery.

Manuscript submitted to ACM

Manuscript submitted to ACM

We treat this cyclicity as what it is: a structural data-quality defect in a *derived* data artifact. The preference graph is not raw data collected from the world; it is manufactured by a specific aggregation procedure applied to upstream ranker scores, and its consistency properties are a direct consequence of that procedure. Viewed this way, a cyclic preference graph is analogous to other well-studied information-quality problems in derived data—contradictory records, non-integrable schemas, or inconsistent provenance chains—in that the defect is measurable, its severity depends on how the data was constructed, and it invites a natural remediation step. Feedback-arc-set (FAS) repair is exactly such a remediation step for preference graphs: it removes the minimum-weight set of edges needed to make the graph acyclic, producing a graph that is, by construction, more internally consistent [1, 9].

The unresolved question is whether this kind of structural repair is worth performing from the standpoint of the information a retrieval system ultimately delivers to a user. Structural consistency and downstream retrieval quality are related but conceptually distinct notions of “quality.” A repaired graph is more internally consistent by the metric that defines the repair, but consistency is not the same property that a ranked-retrieval metric such as normalized discounted cumulative gain (nDCG) measures. It is possible for a repair intervention to measurably improve graph-level consistency while leaving the final ranked list unchanged, or even making it worse, if the edges it removes happen to be uninformative or misleading with respect to the relevance judgments a retrieval system is ultimately evaluated against. Prior work on rank aggregation and feedback-arc-set methods has largely emphasized the algorithmic and structural side of this problem—how to find a good repair, and how to bound its approximation quality relative to a graph-internal objective [1, 5, 6, 9]—without systematically testing whether the resulting structural improvement transfers to the retrieval-quality outcome that end users experience. This leaves a gap: it is not enough to show that a repair procedure exists and reduces a structural inconsistency score; a data-quality account of preference-graph repair also has to characterize when, and how reliably, that structural improvement translates into better ranked output, and to say plainly when it does not.

This paper closes that gap empirically. We study preference-graph inconsistency as a measurable data-quality dimension and evaluate feedback-arc-set repair as a candidate data-quality improvement technique, across four public retrieval benchmarks (SciDocs, FiQA, HotpotQA, and BRIGHT) and three vote-extraction regimes that construct preference graphs from the same underlying ranker scores under different retention rules for directional evidence [3, 12, 15, 19]. This design lets us separate three choices that are often bundled together in reranking pipelines but that have distinct consequences: how permissively pairwise votes are retained during graph construction, whether the resulting graph is repaired before scoring, and which graph-scoring rule is used to turn the (repaired or unrepaired) graph into a ranking. Because vote-extraction regime alone determines whether a graph has any cyclic structure for repair to act on, we report every structural and retrieval outcome broken out by regime rather than pooled, so that a null result in a near-acyclic regime is not mistaken for a failure of the repair procedure itself.

Two results follow from this design. First, vote-extraction regime, not the repair step, is the dominant driver of preference-graph inconsistency: a conservative, majority-style regime keeps graphs almost entirely acyclic on every dataset we test, while a permissive regime that retains every ranker-supported directional vote produces substantially cyclic graphs with much larger strongly connected components. Second, and centrally, we find that structural improvement from repair and downstream retrieval improvement are decoupled in the large majority of the conditions we test. When the underlying graph is already near-acyclic, repair is retrieval-inactive by construction, because there is negligible conflicting edge weight for it to remove. Even in the one regime where graphs are substantially cyclic, the retrieval effect of repair is heterogeneous across datasets and, in three of four cases, remains compatible with no change once bootstrap uncertainty is taken into account. The remaining cell, HotpotQA under *ms1*, has a positive mean

with interval $[0, 0.0405]$; because the lower endpoint is exactly zero and the cell is based on 52 queries, we treat it as bounded and dataset-specific rather than as confirmatory evidence of a general gain. We treat these null and mixed outcomes as part of the contribution, not as a shortfall to be explained away: identifying the conditions under which a data-quality intervention does *not* transfer to task-level utility is itself the actionable finding a practitioner needs before deciding whether to invest in graph repair for a given pipeline.

We are equally careful about scope. The main evidence base uses mechanical BM25/TF-IDF/MiniLM score files, and we supplement it with bounded real-LLM checks rather than presenting those API-based runs as equal-scale replication. The greedy repair heuristic used in the main experiments is compared against an exact-for-small-components variant that solves small strongly connected components exactly under the stated FAS objective and falls back to greedy on larger ones; stronger optimization of that structural objective does not change the retrieval conclusion. We do not restrict the comparison to the repaired-versus-unrepaired pair alone; we additionally evaluate repaired hybrids against simple fixed baselines such as reciprocal rank fusion (RRF) and CombSUM, and we examine whether fusion can suppress a graph-level change before it reaches the final ranking. Finally, where a graph-based diagnostic such as backward-edge weight is computed against the same relevance judgments used for retrieval evaluation, we flag that dependence explicitly rather than presenting it as independent corroboration.

Framed this way, the contribution of this paper is diagnostic and curatorial rather than algorithmic. We do not propose a new ranking algorithm, and we do not claim that feedback-arc-set repair is a generally superior reranking strategy; the evidence does not support that claim. Instead, we contribute: (1) a systematic, regime-stratified measurement of preference-graph structural inconsistency across four benchmarks and 1,006 query×regime evaluation records (Table 2); (2) a bootstrap-quantified account of when feedback-arc-set repair does and does not change downstream retrieval quality, including the single cell with a positive mean and lower interval endpoint at 0.0; (3) a six-class rule-based failure taxonomy that explains *why* repair is retrieval-inactive, neutral, or harmful in specific, interpretable terms, rather than reporting only aggregate null results; (4) a pooled comparison showing that simple, fixed aggregation baselines are competitive with or superior to repaired graph-hybrid rankings, which bears directly on whether structural repair is a good investment relative to simpler alternatives. Together, these contributions position preference-graph consistency as a data-quality problem to be measured and diagnosed on its own terms, with retrieval effectiveness treated as a separate, empirically evaluated outcome rather than an assumed consequence of structural repair.

The remainder of the paper is organized as follows. Section 2 (Background and Related Work) positions this study relative to data-quality frameworks, rank aggregation and preference modeling, and rank fusion. Section 3 (Methodology) formalizes preference graphs, the structural data-quality metrics we measure, the feedback-arc-set repair operator, and the ranking-extraction rules applied before and after repair. Section 4 (Experimental Setup) describes the four-dataset protocol, vote-extraction regimes, baselines, and evaluation methodology, including the bootstrap procedure used throughout. Sections 5–7 report structural results, the repaired-versus-unrepaired retrieval comparison with bootstrap confidence intervals, and the failure taxonomy, respectively. Section 8 reports the bounded real-LLM validation, and Section 9 summarizes runtime and memory implications. Section 10 introduces the CARB benchmark. Section 11 discusses the findings, Section 12 states limitations, and Section 13 concludes; a final section states data and reproducibility availability.

2 BACKGROUND AND RELATED WORK

Data-quality research distinguishes intrinsic properties of a data artifact from contextual, fitness-for-use assessments of whether that artifact supports its downstream task [11, 18]. We adopt that distinction here. A preference graph is

not raw observational data; it is a derived artifact produced by aggregating upstream ranking signals. We therefore treat cyclicity and related graph properties as structural data-quality attributes of that derived representation, while treating retrieval effectiveness as a separate task-level outcome that must be measured rather than assumed.

Preference graphs and pairwise-comparison ranking are standard in rank aggregation [1, 5, 13]. Directed cycles indicate that no single total order can satisfy every retained pairwise preference simultaneously, and feedback-arc-set (FAS) formulations provide a classical way to restore acyclicity by removing a minimum-weight set of edges [1, 9]. Related lines of work study probabilistic ranking from pairwise outcomes, such as Rank Centrality [13], and inconsistency decomposition, such as HodgeRank, which uses cyclic components of pairwise data as a diagnostic signal [8]. Our use of cyclicity is closer to this diagnostic tradition than to an optimization paper proposing a new ranker: the question here is not whether a graph can be made more internally consistent, but whether such structural cleanup propagates to better retrieval.

This distinction also separates the present study from work on rank fusion and LLM reranking. Rank fusion methods such as CombSUM and reciprocal rank fusion [4, 7] collapse multiple signals directly into a final score without reasoning over graph structure; they are therefore strong and practically important baselines for any graph-repair pipeline. Recent LLM reranking work uses pointwise, pairwise, and listwise prompting to obtain relevance preferences or direct reorderings from language models [14, 16]. Our bounded real-LLM experiments use such judgments only as alternate preference sources to test the scope of the main finding; we do not propose a new LLM reranker, nor do we pool those smaller API-based runs with the main mechanical-vote evaluation.

The paper’s main departure from prior rank-aggregation, reranking, and inconsistency-modeling work is therefore evaluative. Prior studies typically optimize a ranking objective, improve a graph-internal consistency criterion, or show that a fusion rule works well as a ranker. We instead ask a narrower data-quality question: when a derived preference graph is structurally repaired under its own objective, does that intervention improve the retrieval metric of interest? Because the repair objective does not directly optimize nDCG, no monotonic relationship is guaranteed. The rest of the paper keeps these two quality dimensions separate on purpose.

3 METHODOLOGY

This section defines the preference-graph construction procedure, the structural data-quality metrics computed on it, the feedback-arc-set repair operator, and the ranking-extraction rules applied to a graph before and after repair. We keep these four stages notationally and conceptually distinct throughout, because conflating them is a recurring source of ambiguity in prior discussions of “graph repair for reranking”: repair changes which edges a graph contains; extraction turns a (fixed) set of edges into a ranking. The same extraction rule is applied to both the unrepaired and the repaired graph, so any difference in the resulting ranking is attributable to repair and not to a change in the extraction rule itself. Table 1 summarizes notation used in this section and Section 4.

3.1 Vote Extraction and Graph Construction

Graph construction begins from per-ranker document scores, not from pairwise judgments collected directly. For each query q and each constituent ranker s , and for every unordered candidate pair $\{u, v\} \subset D_q$, ranker s casts a directional *vote* for whichever of u, v it scores higher, with vote weight equal to the ranker’s own score margin $|score_s(u) - score_s(v)|$; a ranker abstains from a pair if either document is missing from its score list, or if its margin on that pair falls below a fixed per-vote threshold. Votes are then aggregated per unordered pair and per direction: for a candidate direction (u, v) , let $support(u, v)$ be the number of rankers voting for that direction and $margin_sum(u, v)$ the sum of

Table 1. Notation used in Sections 3 and 4.

Symbol	Meaning
q, D_q	Query and its candidate document set
$G_q = (V_q, E_q, w_q)$	Preference graph for query q ($V_q = D_q$)
$\tilde{G}_q$	Repaired (acyclic) preference graph
$u > v$	Pairwise preference: u ranked ahead of v
r	Vote-extraction regime, $r \in \{\text{ms2}, \text{ms1}, \text{ms1_drop_mutual}\}$
F	A feedback arc set (edge subset removed by repair)
$\deg^+(u), \deg^-(u)$	Out-degree, in-degree of node u in a graph
$s_{\text{cop}}, s_{\text{bal}}$	Copeland score, weighted balance score
$\hat{s}(\cdot)$	Min-max normalization of a score vector to $[0, 1]$
α	Hybrid mixing weight for the graph-derived component
π	A ranking (permutation) of V_q
$\text{BEW}(\pi, G)$	Backward-edge weight of π relative to graph G
$\text{PIC}(\pi, G)$	Pairwise inconsistency count (unweighted analogue)
k	nDCG cutoff

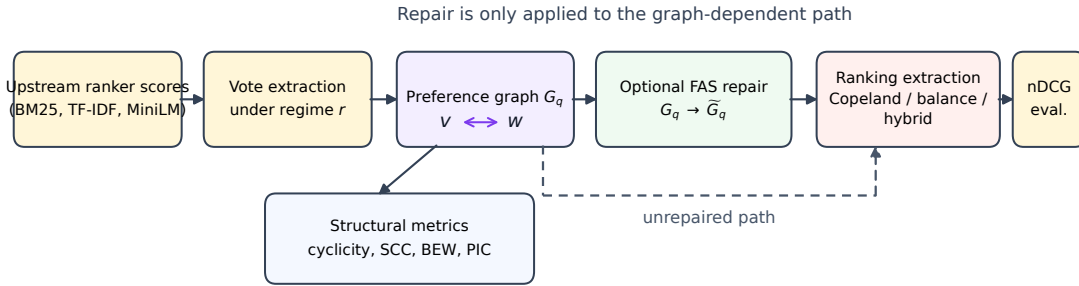


Fig. 1. Preference-graph data-quality evaluation pipeline: upstream ranker scores $\rightarrow$ vote extraction under a regime $r \rightarrow$ preference graph $G_q \rightarrow$ structural metrics $\rightarrow$ optional FAS repair producing $\tilde{G}_q \rightarrow$ ranking extraction (Copeland / balance / hybrid) on either G_q or $\tilde{G}_q \rightarrow$ nDCG evaluation.

their margins. A directed edge (u, v) is retained in E_q , with weight $w_q(u, v) = \text{margin_sum}(u, v)$, only if

$$\text{support}(u, v) \geq \tau_{\text{sup}} \quad \text{and} \quad \text{margin_sum}(u, v) \geq \tau_{\text{marg}}, \quad (1)$$

where the thresholds τ_{sup} (minimum support) and τ_{marg} (minimum aggregate margin) are fixed per vote-extraction regime r (Section 3.2). Because both directions of an unordered pair are evaluated independently under Eq. (1), a pair can retain edges in *both* directions when rankers disagree and each direction separately clears the threshold; this is the mechanism by which cycles enter G_q . Candidate documents are pooled across rankers by reciprocal rank fusion [4] before vote extraction, so D_q is a shared set regardless of which individual rankers surface which documents.

3.2 Vote-Extraction Regimes

We construct three graphs per query from the same underlying ranker scores, differing only in $(\tau_{\text{sup}}, \tau_{\text{marg}})$ and one post-processing step, which we refer to as *regimes*:

- **ms2** (conservative): $\tau_{\text{sup}} = 2$, $\tau_{\text{marg}} = 0.1$. A direction is retained only with majority ranker support and a non-trivial aggregate margin, which suppresses most single-ranker disagreements.
- **ms1** (permissive): $\tau_{\text{sup}} = 1$, $\tau_{\text{marg}} = 0.0$. Every ranker-supported direction is retained, so both directions of a pair can coexist whenever rankers disagree.
- **ms1_drop_mutual**: built from the ms1 output by deleting *all* edges for any unordered pair that retained both directions, treating mutual disagreement as unresolved rather than keeping either arc.

These three regimes hold the upstream ranker scores fixed and vary only the rule by which disagreement becomes graph structure, which lets us attribute differences in graph cyclicity (Section 3.3) to the extraction rule rather than to the underlying evidence.

3.3 Structural Data-Quality Metrics

We measure four structural properties of a preference graph G_q : whether it is acyclic (a binary indicator), the size of its largest strongly connected component (a graph with no cycles has all components of size 1), its edge count, and, for repair specifically, the total weight of edges a repair operation removes (Section 3.4). We additionally define two metrics that relate a graph or ranking to an external reference ranking π_{ref} (in our experiments, a ranking derived from relevance judgments): the **backward-edge weight**

$$\text{BEW}(\pi_{\text{ref}}, G) = \sum_{(u,v) \in E: \pi_{\text{ref}}(v) < \pi_{\text{ref}}(u)} w(u,v), \quad (2)$$

summing the weight of every edge (u,v) (asserting $u > v$) that π_{ref} contradicts by ranking v ahead of u , and the **pairwise inconsistency count**

$$\text{PIC}(\pi_{\text{ref}}, G) = |\{(u,v) \in E : \pi_{\text{ref}}(v) < \pi_{\text{ref}}(u)\}|, \quad (3)$$

the unweighted count of the same contradictions. BEW and PIC are graph-versus-reference diagnostics, not properties of a graph alone; when the reference ranking is derived from the same relevance judgments used to compute nDCG (Section 3.6), this introduces a circularity that we flag explicitly rather than treat as an independent validation of structural repair (Section 12, forward reference).

3.4 Graph Repair

Repair is a single, well-defined operation on a graph: given G_q , find a feedback arc set $F \subseteq E_q$ minimizing removed weight subject to acyclicity of the remainder,

$$F^* = \arg \min_{F \subseteq E_q : (V_q, E_q \setminus F) \text{ acyclic}} \sum_{(u,v) \in F} w_q(u,v), \quad (4)$$

and return $\tilde{G}_q = (V_q, E_q \setminus F^*, w_q)$. If G_q is already acyclic, $F^* = \emptyset$ and $\tilde{G}_q = G_q$. Solving Eq. (4) exactly is NP-hard in general [1]; our main experiments use a greedy heuristic with no approximation guarantee, not an exact solver: repeatedly locate any directed cycle, remove its minimum-weight edge, and continue until no cycle remains. This is a simple cycle-peeling procedure, not a claim of optimality, and we compare it against stronger repair variants in Section 4 to test whether the specific choice of heuristic changes our conclusions. We report the total weight $\sum_{(u,v) \in F^*} w_q(u,v)$ removed by repair as a direct measure of how much repair had to do on a given graph; when this quantity is zero, repair is structurally inactive and $\tilde{G}_q = G_q$ by construction.

Repair is defined purely in terms of w_q and does not reference any external relevance signal. This is the structural fact underlying the Background section’s argument (Section 2): nothing in Eq. (4) constrains whether the edges it removes are relevant, irrelevant, or misleading with respect to a downstream retrieval metric.

3.5 Ranking Extraction

Ranking extraction is a separate operation that consumes a graph (either G_q or $\tilde{G}_q$) and produces a ranking π of V_q ; the same extraction rule is applied regardless of which graph it receives, so repair and extraction can be varied independently. We use two graph-derived component scores. The **Copeland score** is out-degree minus in-degree,

$$s_{\text{cop}}(u) = \deg^+(u) - \deg^-(u), \quad (5)$$

counting net wins irrespective of edge weight. The **weighted balance score** is the signed sum of incident edge weight,

$$s_{\text{bal}}(u) = \sum_{(u,v) \in E} w(u,v) - \sum_{(v,u) \in E} w(v,u). \quad (6)$$

Both scores are well-defined on a graph with cycles—neither requires acyclicity, since they are computed from local degree and weight sums rather than from a topological order. This is what makes it possible to compute s_{cop} and s_{bal} on the *unrepaired* graph G_q directly, without first forcing an arbitrary tie-break on its cycles.

The primary ranking methods we evaluate are hybrids that combine one of these two component scores with a query’s prior ranking, obtained by reciprocal rank fusion [4] of the constituent rankers’ score lists. Writing $\hat{s}(\cdot)$ for min–max normalization of a score vector to $[0, 1]$ over the candidate set (so components on different native scales are comparable), the hybrid score is

$$s_{\text{hybrid}}(u) = \hat{s}_{\text{prior}}(u) + \alpha \hat{s}_{\text{comp}}(u), \quad \text{comp} \in \{\text{cop}, \text{bal}\}, \quad (7)$$

with $\alpha = 0.3$ throughout the main experiments, and the extracted ranking π sorts V_q by descending s_{hybrid} , breaking exact ties deterministically by document id. Applying Eq. (7) with s_{comp} computed on G_q yields the *unrepaired* hybrid ranking for that component; applying it with s_{comp} computed on $\tilde{G}_q$ yields the *repaired* hybrid ranking. The two rankings differ only through s_{comp} , since $\hat{s}_{\text{prior}}$ and α are identical in both cases—so any difference in downstream nDCG between the repaired and unrepaired hybrid is attributable to repair changing s_{comp} , not to a change in how the ranking is extracted from a score.

3.6 Retrieval Metric

Retrieval quality is measured by normalized discounted cumulative gain at cutoff k . For a ranking π and a relevance map $\text{rel} : D_q \rightarrow \mathbb{Z}_{\geq 0}$,

$$\text{nDCG}@k(\pi) = \frac{\sum_{i=1}^k \frac{2^{\text{rel}(\pi_i)} - 1}{\log_2(i+1)}}{\sum_{i=1}^k \frac{2^{\text{rel}(\pi_i^{\text{ideal}})} - 1}{\log_2(i+1)}}, \quad (8)$$

where π_i is the document at rank i in π and π^{ideal} sorts all judged documents by descending relevance; $\text{nDCG}@k$ is defined as 0 when the ideal-ranking denominator is 0 (no relevant documents judged for the query). Eq. (8) is the

Table 2. Four-dataset mechanical-vote protocol. “top- n ” is the per-ranker retrieval depth before pooling; k is both the candidate-pool size and the nDCG cutoff. “Usable queries” lists realized counts under ms2/ms1/ms1_drop_mutual.

Dataset	Judgments	top- n / k	Source	Usable queries
SciDocs	binary qrels	50 / 20	SciDocs + BEIR	119 / 120 / 120
FiQA	positive-only judged qrels	50 / 20	BEIR-FiQA	117 / 120 / 120
HotpotQA	binary qrels	35 / 10	HotpotQA	52 / 52 / 52
BRIGHT	binary qrels	50 / 20	BRIGHT	34 / 50 / 50

only retrieval-effectiveness metric used for the main comparisons in this paper; it is entirely independent of the graph-internal metrics of Section 3.3, which is the property that makes it possible to ask, rather than assume, whether repair helps retrieval.

4 EXPERIMENTAL SETUP

4.1 Datasets

We use four public retrieval benchmarks spanning distinct domains: SciDocs (scientific document retrieval) [3], FiQA (financial opinion and question answering) [12], HotpotQA (multi-hop question answering) [19], and BRIGHT (reasoning-intensive retrieval) [15]. SciDocs, FiQA, and HotpotQA additionally appear in the BEIR zero-shot retrieval benchmark suite [17]; BRIGHT is a more recent addition designed specifically to stress-test retrieval under reasoning-heavy queries. Eligible queries are sampled without replacement from the processed benchmark exports with seed 42 up to per-dataset budgets in the evaluation script. Table 2 reports the resulting protocol parameters. “Usable queries” counts can differ slightly across regimes because eligibility and vote extraction are both applied per query: a query can enter the evaluation pool but still contribute no votes under a stricter regime if no candidate pair clears that regime’s retention thresholds.

4.2 Rankers and Candidate Pooling

Upstream score files come from three mechanical rankers: an in-house Okapi BM25 implementation ($k_1 = 1.5$, $b = 0.75$), an in-house cosine TF-IDF ranker with log-TF and smoothed IDF weighting, and a MiniLM dense encoder (sentence-transformers/all-MiniLM-L6-v2) scored by cosine similarity. For each query, the candidate pool D_q is formed by reciprocal rank fusion [4] with the standard constant $k = 60$ over the three rankers’ top- n lists, then truncated to the per-dataset pool size in Table 2. Votes are extracted over that fixed pool as described in Section 3.1, with margin weighting, abstention when a per-ranker score margin is below 0.05, and abstention when a ranker lacks a score for either candidate in a pair. We do not vary the ranker family across regimes or datasets: ms2, ms1, and ms1_drop_mutual are three views of the *same* three rankers’ scores, so any difference between regimes reflects the vote-retention rule, not a change in evidence source. We return to the scope this narrows the study to in Section 12: the main evidence base uses mechanical, score-derived votes rather than direct human or LLM pairwise judgments.

4.3 Baselines

Table 3 lists the ranking methods evaluated on the four-dataset evaluation. The methods of primary interest are the four repaired/unrepaired hybrid pairs (Section 3.5, Eq. (7)), since these isolate the marginal effect of repair while holding the prior ranking, the component score family, and α fixed. The stored four-dataset outputs use a single fixed mixing weight, $\alpha = 0.3$; no held-out validation tuning for that parameter is recorded for this protocol, so we treat it as an

Table 3. Ranking methods evaluated in the four-dataset evaluation. “Uses graph?” indicates whether the method’s score depends on the preference graph (as opposed to ranker scores alone); “Repair variants?” indicates whether the method is evaluated in both unrepaired and repaired form.

Method	Uses graph?	Repair variants?
Prior (RRF of BM25/TF-IDF/MiniLM)	No	—
Reciprocal rank fusion (RRF)	No	—
CombSUM	No	—
Borda-count fusion	No	—
Markov (Rank Centrality-style)	Yes	Yes
Copeland hybrid (Eq. (7), comp. = Copeland)	Yes	Yes
Balance hybrid (Eq. (7), comp. = balance)	Yes	Yes

implementation choice rather than an optimized setting. We additionally evaluate the prior ranking alone, and a set of established fusion and graph-aggregation baselines that do not depend on repair at all: reciprocal rank fusion [4], CombSUM [7], a Borda-count rank-aggregation baseline (points assigned by rank position, summed across rankers), and a Rank-Centrality-style Markov chain ranker [13] computed on the (unrepaired or repaired) preference graph. CombSUM fuses per-ranker scores by summing, for each candidate document, a per-(query, ranker) min–max normalization of that ranker’s score to $[0, 1]$; a ranker that did not score a given document contributes 0 to that document’s fused score, and exact ties are broken first by the smallest rank the document achieved in any individual ranker’s own list, then by document id. The min–max normalization step is an adaptation required by this study’s heterogeneous ranker scales (BM25, TF-IDF, and dense MiniLM cosine scores) and is not part of the original 1994 definition, which we note explicitly rather than presenting an adapted baseline as an unmodified reproduction. These baselines answer a question distinct from the repair-effect question: whether graph repair combined with hybrid fusion is competitive with substantially simpler aggregation rules that never inspect graph structure at all.

The three baselines marked “No” under “Uses graph?” in Table 3—the prior ranking, reciprocal rank fusion, and CombSUM (Borda-count fusion likewise)—depend only on the constituent rankers’ score files and are therefore invariant to vote-extraction regime: their score for a given query is identical whether that query’s graph is built under `ms2`, `ms1`, or `ms1_drop_mutual`, since regime only changes how the graph is constructed from those same scores. When we report these baselines broken out by regime alongside the graph-dependent methods, the repeated entries reflect the same underlying score fused once and aligned across regimes for comparability, not independent reruns or additional evidence; any aggregate statistic computed over regime-labeled rows for a graph-independent baseline should be read with this in mind.

A separate evaluation protocol (Section 6) assesses a wider pooled grid of twelve methods, including the hybrids above, on a different query pool assembled for failure-taxonomy analysis. We keep this pooled comparison clearly labeled as a distinct protocol throughout the paper rather than merging its numbers with the four-dataset evaluation above: the two protocols share method implementations but not query sets, and reporting them as though they were one evaluation would overstate the evidence either provides alone.

4.4 Repair Variants Compared

Table 4 lists the repair procedures we compare, both fully reproducible from the archived code and stored outputs used in this study. The *greedy* heuristic (Eq. (4) solved by cycle-peeling, Section 3.4) is used throughout the main experiments.

Table 4. Repair procedures compared (see text for reproducibility notes).

Procedure	Scope
Greedy (cycle-peeling)	All queries; the repair method used throughout the main experiments
Exact-for-small-components (brute-force, SCCs ≤ 10 nodes) + greedy fallback above that size	All queries

The second procedure applies an exact solver—exhaustive enumeration of node orderings—to any strongly connected component (SCC) with at most 10 nodes, and falls back to the same greedy heuristic for larger components within the same query; we call this combined procedure *exact-for-small-components*, since it is exact only on the (typically majority) share of SCCs at or below that size threshold and is not a globally exact solver for the whole graph. Both procedures cover the full four-dataset evaluation (all four datasets, all three regimes), not a subsampled slice.

This comparison is sufficient to test whether improving the graph-internal structural objective (Eq. (4)) more thoroughly than the greedy heuristic translates into a retrieval-quality gain. Under the stated FAS objective, a feasible repair is any edge-removal set that makes the graph acyclic, and lower removed weight is better among such repairs. Exact-for-small-components is exact only on SCCs of at most 10 nodes and falls back to greedy above that size, so it is not a globally exact solver for the whole graph. The purpose of this comparison is therefore narrower than “more aggressive repair”: it tests whether solving the same graph-internal objective more accurately on small SCCs changes the retrieval conclusion of Section 6.

4.5 Statistical Testing: Paired Bootstrap

Every repaired-versus-unrepaired comparison in Table 8 and Figure 4 is evaluated with a paired, per-query percentile bootstrap. For a method pair (e.g., repaired and unrepaired Copeland hybrid) and a set of n queries for which both methods produced an $\text{nDCG}@k$ value, let $\delta_i = \text{nDCG}@k_i^{\text{repaired}} - \text{nDCG}@k_i^{\text{unrepaired}}$ for query $i = 1, \dots, n$. We draw $B = 2,000$ bootstrap resamples, each obtained by sampling n query indices independently and uniformly with replacement from $\{1, \dots, n\}$ and averaging the corresponding δ_i ; writing $\bar{\delta}^{(b)}$ for the mean of resample b , the reported interval is

$$\left[\bar{\delta}^{(\lceil 0.025(B-1) \rceil)}, \bar{\delta}^{(\lceil 0.975(B-1) \rceil)} \right], \quad (9)$$

the empirical 2.5th and 97.5th percentiles of the sorted resample means $\bar{\delta}^{(1)} \leq \dots \leq \bar{\delta}^{(B)}$, alongside the observed mean $\bar{\delta} = \frac{1}{n} \sum_i \delta_i$. We use this percentile interval throughout rather than a normal-approximation interval because many of the per-query delta distributions are highly non-normal: in near-acyclic regimes (Section 3.2), $\delta_i = 0$ for every query, and the resample distribution is a point mass at zero rather than anything a normal approximation would describe well. All confidence intervals reported in this paper use $B = 2,000$ unless stated otherwise, with the stored four-dataset tables generated under seed 42. These intervals are descriptive and are not adjusted for multiplicity across the 24 repaired-versus-unrepaired cells.

Section 6’s pooled baseline comparison uses a different resampling unit. Its underlying corpus contains 1,020 query×regime records but only 342 independent (dataset, query) clusters, because a single query may contribute multiple regime-labeled rows and some baselines are regime-invariant by construction. For Figure 5, we therefore report marginal method-wise percentile intervals from a clustered bootstrap that resamples those 342 query clusters with replacement, carrying all associated rows together. The plotted intervals use 1,000 resamples with seed 13 and should be read as descriptive marginal uncertainty for each method mean, not as paired tests between methods.

4.6 Implementation, Hardware, and Resource Measurement

The full evaluation procedure (vote extraction, graph construction, greedy repair, ranking extraction, nDCG evaluation, and bootstrap analysis) is implemented in Python (3.11/3.12) using `networkx` for graph operations, and is reproducible end to end from the archived code (Section 14). Wall-clock timing is measured per pipeline stage with `time.perf_counter()` around each stage in isolation; we report per-stage means over queries rather than a single end-to-end number, since the stages have very different and dataset-dependent costs (repair dominates on large, cyclic graphs; ranking extraction and evaluation are comparatively cheap at every scale we test). Memory is measured as the maximum of the process’s resident set size (RSS) sampled immediately before and immediately after a stage; this is a coarse before/after snapshot, not instrumented peak-memory tracking (e.g., `tracemalloc`), and should be read as an approximate, directional indicator rather than a precise memory profile. All runtime and memory measurements in this paper were collected on a single, unbenchmarked local development machine rather than a controlled or standardized hardware configuration; we report relative comparisons across methods run on the same machine rather than absolute figures intended to generalize to other hardware. The codebase additionally contains a complete, self-contained integer-programming formulation of Eq. (4) that requires a commercial mixed-integer solver license to run; it was not used to produce any result reported in this paper. Both procedures in Table 4 require neither a commercial solver license nor any dependency outside the archived codebase.

4.7 Real-LLM Evidence: Scope and Role

In addition to the mechanical three-ranker evaluation described above, we evaluate a small, bounded sample of real large-language-model judgments (pairwise, pointwise, and listwise) on subsets of SciDocs, HotpotQA, and FiQA, using the same repaired-versus-unrepaired comparison structure. We describe the primary paired OpenAI pilot in Section 8, and we keep it separate from a second, protocol-distinct multi-provider failure-mining corpus with Cohere/Azure pairwise judgments on FiQA, HotpotQA, and BRIGHT. We flag this scope here because it is easy to conflate any real-LLM evidence with the main mechanical-vote evaluation if the protocols are not kept separate. The real-LLM sample is one to two orders of magnitude smaller than the main evaluation (tens of queries per dataset rather than dozens to low hundreds), and is reported throughout this paper as a bounded check on whether the regime-conditional pattern of Sections 5 and 6 persists under a qualitatively different preference source—not as independent confirmatory evidence at the same scale or standard of certainty as the main evaluation.

4.8 LLM Providers and Inference Configuration

Table 5 summarizes the API-based runs that inform the manuscript. The primary paired evidence is an OpenAI pairwise pilot using `gpt-4o-mini`. Separate OpenAI pointwise and listwise runs are used only as auxiliary scope checks. The protocol-distinct corroborative corpus uses Cohere and Azure OpenAI pairwise judgments over a different method pair (repaired vs. unrepaired Markov), and is therefore not pooled with the primary pilot. The same experiment logs also

Table 5. API-based experiments used in the manuscript or supporting scope checks.

Provider / platform	Model	Mode	Datasets / role
OpenAI API	gpt-4o-mini	Pairwise	Primary paired pilot on SciDocs, HotpotQA, and FiQA.
OpenAI API	gpt-4o-mini	Pointwise / listwise	Auxiliary prompt-style scope check on SciDocs and HotpotQA.
Cohere API	command-r-plus-08-2024	Pairwise	Protocol-distinct corroborative corpus on FiQA, HotpotQA, and BRIGHT.
Azure OpenAI	gpt-4.1-mini	Pairwise	Protocol-distinct corroborative corpus on FiQA, HotpotQA, and BRIGHT.

Table 6. Structural metrics by dataset and vote-extraction regime. Cyclic% is the share of query graphs containing at least one directed cycle; Largest SCC is the mean size of the largest strongly connected component.

Dataset	Regime	Cyclic%	Largest SCC
SciDocs	ms2	0.0	1.00
SciDocs	ms1	87.5	9.33
SciDocs	ms1_drop_mutual	0.0	1.00
FiQA	ms2	0.0	1.00
FiQA	ms1	95.0	12.51
FiQA	ms1_drop_mutual	0.0	1.00
HotpotQA	ms2	0.0	1.00
HotpotQA	ms1	51.9	2.52
HotpotQA	ms1_drop_mutual	0.0	1.00
BRIGHT	ms2	0.0	1.00
BRIGHT	ms1	60.0	6.48
BRIGHT	ms1_drop_mutual	6.0	1.40

record Gemini setup attempts, but the versioned failure-mining corpus used here contains no usable Gemini records, so Gemini is not treated as analyzed evidence in this revision.

Across these runs, decoding is deterministic (temperature 0.0). The paired OpenAI pilot targets 50 SciDocs queries, 20 HotpotQA queries, and 20 FiQA queries, of which 10 FiQA queries completed successfully. The auxiliary pointwise/listwise check covers 20 SciDocs queries and 10 HotpotQA queries per prompt style. Each protocol-distinct corroborative provider contributes 200 query×regime records across FiQA, HotpotQA, and BRIGHT. The paired OpenAI pilot uses top- $k = 15$ candidate pools, seed 42, no candidate-order debiasing, and pairwise responses constrained to a one-letter A/B judgment with up to four retries and exponential backoff on API failures; ambiguous responses fall back to the parser’s default A-label. The failure-mining corpus uses the same A/B parsing but enables symmetric A→B and B→A prompting to reduce position bias. Detailed prompts and API logs are better suited to supplementary material than to the main paper, so we summarize only the settings needed to interpret scope and comparability here.

5 STRUCTURAL DATA QUALITY RESULTS

Table 6 reports the structural metrics defined in Section 3.3 for all four datasets under all three vote-extraction regimes.

The regime contrast is stark and uniform across datasets. Under ms2, every dataset is effectively acyclic (0% cyclic queries, largest SCC = 1.0); under ms1, cyclic-query prevalence rises to 51.9–95.0% with largest SCC 2.5–12.5, an order-of-magnitude structural change from the same underlying ranker scores. ms1_drop_mutual restores near-acyclicity

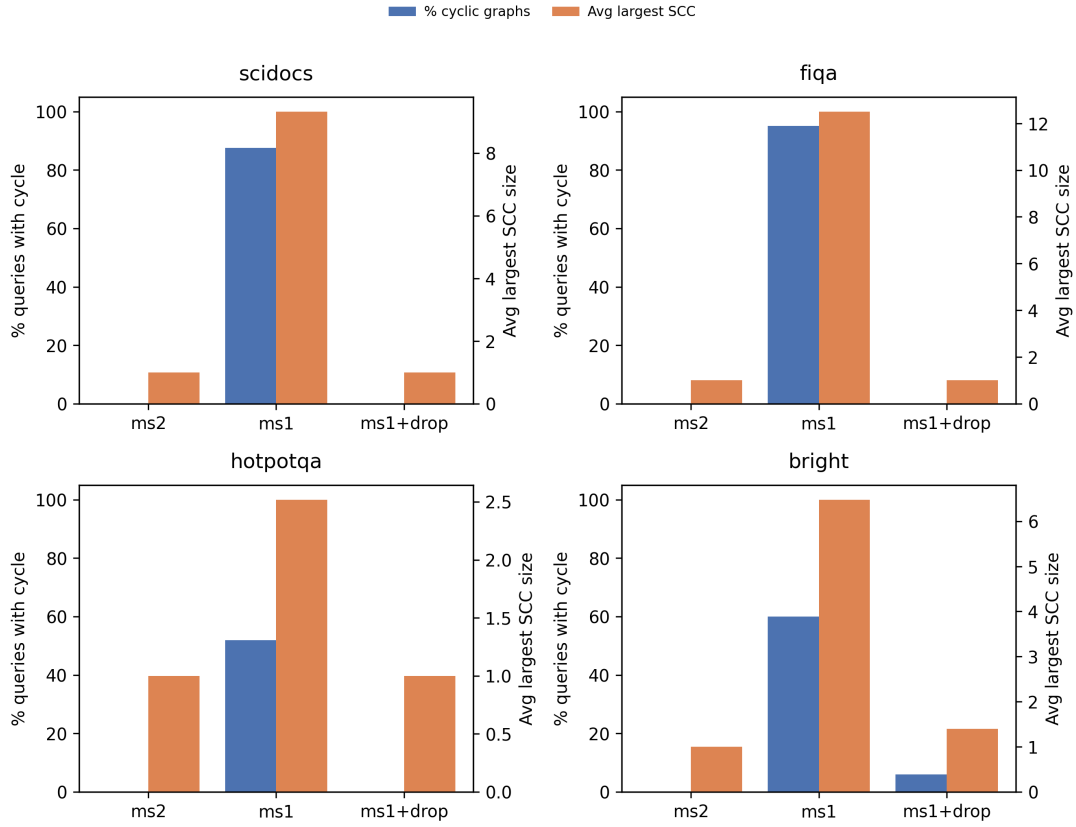


Fig. 2. Percentage of cyclic query graphs and mean largest strongly connected component size, by dataset and vote-extraction regime. ms2 and ms1_drop_mutual are near-acyclic on every dataset; ms1 is substantially cyclic on all four, with HotpotQA showing the mildest effect and FiQA the strongest.

on three of four datasets; BRIGHT retains a small residual (6.0% cyclic, largest SCC 1.4), the one partial exception to an otherwise clean pattern. Figure 2 shows this contrast graphically. Because the three regimes share identical upstream ranker scores (Section 3.2), this difference is attributable to the vote-retention rule alone, not to any change in evidence quality — the central structural finding of this study is that *construction*, not repair, is the dominant lever on preference-graph consistency.

Repair acts only where cyclicity is present, so its structural effect is concentrated in ms1. Table 7 reports mean backward-edge weight (BEW, Eq. (2)) and pairwise inconsistency count (PIC, Eq. (3)) before and after repair, computed against a ranking derived from relevance judgments, together with the mean FAS weight removed (Section 3.4).

Repair reduces PIC on all four datasets (0.5–6.2 fewer inconsistent pairs) with corresponding non-zero FAS weight removed; BEW reductions are smaller in absolute magnitude but consistently positive. Figure 3 shows the BEW pre/post pair graphically (PIC values are in Table 7 only). We emphasize a caveat that is essential to how this result should be read: BEW and PIC are computed against a reference ranking derived from the same relevance judgments later used to compute nDCG@k (Section 3.6). Whenever we describe repair as improving structural consistency, that improvement

Table 7. Mean BEW and PIC before and after FAS repair, and mean FAS weight removed, under ms1 (the only regime with material cyclicity; ms2/ms1_drop_mutual remove ≈ 0 weight by construction).

Dataset	Δ BEW	Δ PIC	Weight removed
SciDocs	0.333	4.25	0.682
FiQA	0.472	6.22	1.099
HotpotQA	0.035	0.50	0.121
BRIGHT	0.234	2.76	0.388

Consistency vs labels: preference graph vs qrels reference ranking

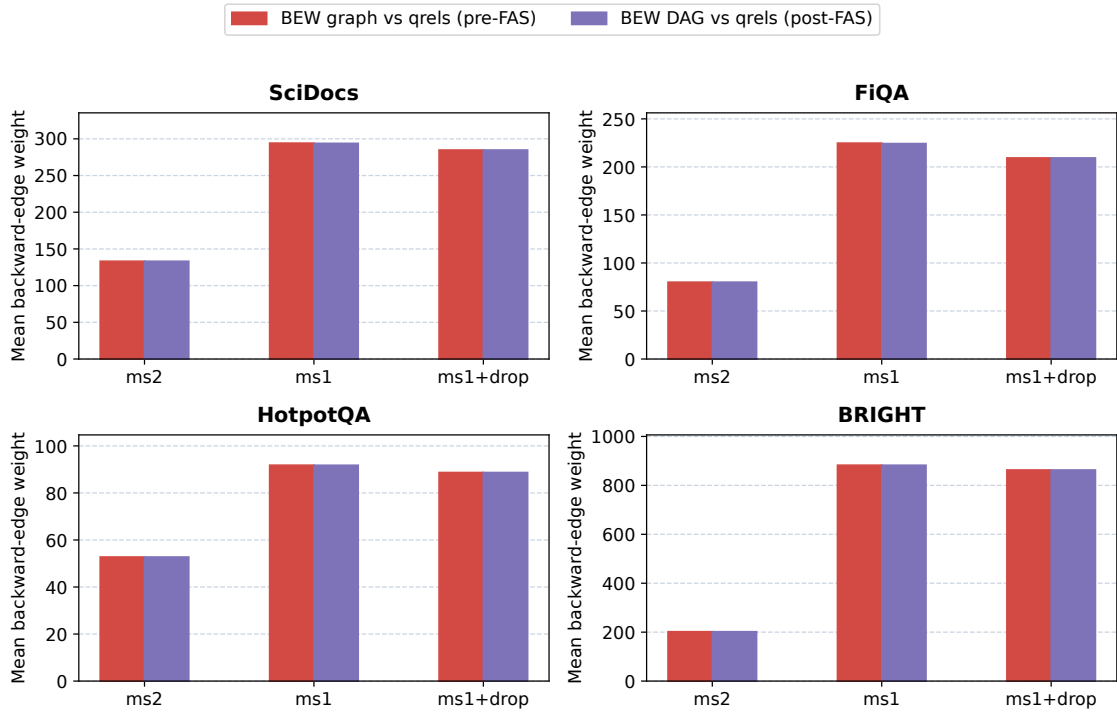


Fig. 3. Mean backward-edge weight (BEW) before and after FAS repair for all four datasets and all three vote-extraction regimes (ms2, ms1, and ms1_drop_mutual); corresponding PIC values are reported in Table 7 rather than plotted here. BEW is computed against the same relevance judgments used for downstream nDCG evaluation (Section 3.6) and should be read as a structural diagnostic rather than an independent validation of repair quality.

is being scored against the same evaluation signal used downstream — a genuine but qrels-anchored diagnostic, not an independent measure of ground truth, and not by itself evidence that a retrieval-quality gain must follow. Sections 6 and 7 test that separate question directly and find that it does not follow uniformly.

6 DOWNSTREAM QUALITY RESULTS: REPAIR VS. RETRIEVAL

Section 5 established that repair measurably improves structural consistency when the graph is cyclic. This section asks the separate question motivating this study: does that structural improvement translate into a retrieval-quality

Table 8. Bootstrap mean $\Delta nDCG@k$ (repaired – unrepaired Copeland hybrid), with 95% CI (2,000 paired query-level resamples, Eq. (9)).

Dataset	Regime	n	Mean Δ	95% CI
SciDocs	ms2	119	0.0	[0, 0]
SciDocs	ms1	120	-0.0001	[-0.0008, 0.0006]
SciDocs	ms1_drop_mutual	120	0.0	[0, 0]
FiQA	ms2	117	0.0	[0, 0]
FiQA	ms1	120	+0.0015	[-0.0005, 0.0042]
FiQA	ms1_drop_mutual	120	0.0	[0, 0]
HotpotQA	ms2	52	0.0	[0, 0]
HotpotQA	ms1	52	+0.0167	[0, 0.0405]
HotpotQA	ms1_drop_mutual	52	0.0	[0, 0]
BRIGHT	ms2	34	0.0	[0, 0]
BRIGHT	ms1	50	+0.00002	[-0.0018, 0.0016]
BRIGHT	ms1_drop_mutual	50	0.0	[0, 0]

gain, measured by $nDCG@k$ (Eq. (8))? We report the paired bootstrap comparison (Section 4.5) for the repaired-versus-unrepaired Copeland and balance hybrids (Eq. (7)), then the pooled comparison against fixed aggregation baselines, then the stronger-repair comparison already introduced in Table 4.

6.1 Repaired vs. Unrepaired Hybrids

Table 8 reports the bootstrap mean $\Delta nDCG@k$ (repaired minus unrepaired) for the Copeland hybrid under all four datasets and three regimes; the corresponding balance-hybrid row is degenerate at exactly [0, 0] in every one of the twelve cells and is summarized rather than tabulated in full. Figure 4 plots all twenty-four cells (Copeland and balance) together.

Twenty of the twenty-four dataset $\times$ regime $\times$ pair cells (Copeland and balance combined) show an interval of exactly [0, 0]: every ms2 and ms1_drop_mutual cell, and every balance-hybrid cell regardless of regime. This is the direct retrieval-level consequence of Section 5’s structural finding – when the graph is already near-acyclic, repair removes negligible edge weight (Table 7), and an intervention with nothing to act on cannot change the ranking it produces. Among the four ms1/Copeland cells, where the graph is substantially cyclic, three intervals straddle zero (SciDocs, FiQA, BRIGHT); one, HotpotQA, has a positive mean with interval [0, 0.0405] – the only cell in the full table whose interval does not extend below zero. Because its lower endpoint is exactly 0.0, the appropriate reading is restrained: repair does not show evidence of harm in this HotpotQA ms1 cell and may help, but the interval does not by itself establish a lower bound above zero. Notably, HotpotQA has the *lowest* ms1 cyclicity (51.9%) of the four datasets (Table 6) – the one non-negative interval does not come from the most structurally inconsistent dataset, which weighs against a simple “more cyclicity implies more benefit from repair” reading of these results.

6.2 Comparison with Fixed Aggregation Baselines

The comparison above isolates the marginal effect of repair while holding fusion and extraction fixed. A separate question is whether the resulting hybrid pipeline is competitive with simple aggregation rules that never inspect graph structure at all. Table 9 reports mean $nDCG@k$ pooled over the 1,020-record failure-mining corpus (Section 4.3) for twelve methods, and Figure 5 visualizes the same pooled ordering with marginal bootstrap intervals.

Table 9. Pooled mean nDCG@k by method, 1,020 query×regime records (failure-mining protocol; Section 4.3).

Method	Mean nDCG
CombSUM	0.462
Reciprocal rank fusion	0.459
Prior only	0.457
Repair-based hybrid ($\alpha = 0.3$)	0.455
Exact-for-small-components hybrid	0.455
Borda-count fusion	0.439
Copeland unrepaired	0.439
Copeland repaired	0.439
Markov repaired	0.435
Markov unrepaired	0.434
Balance hybrid	0.434
Score-sum (graph)	0.433

CombSUM and reciprocal rank fusion — both graph-independent (Section 4.3) — achieve the highest and second-highest pooled mean nDCG, ahead of every graph-based method including the repaired Copeland hybrid and the repair-based hybrid construction evaluated in this paper. The prior ranking alone also exceeds repaired Copeland. This is a direct answer to whether graph repair combined with hybrid fusion is a better retrieval strategy than substantially simpler alternatives: on this pooled comparison, it is not. As noted in Section 4.3, CombSUM, RRF, Borda-count, and the prior ranking are regime-invariant by construction, so their pooled n reflects corpus completeness across regime-labeled rows for the same underlying queries rather than independent per-regime observations; this affects the precision of their own reported estimates. We therefore report Figure 5’s uncertainty using a clustered bootstrap over independent queries rather than treating all 1,020 rows as exchangeable draws. Even with that correction, the main descriptive result is unchanged: graph-independent baselines occupy the top of the pooled point-estimate ranking, and the graph-dependent methods form the lower group.

6.3 Stronger Repair Does Not Change the Conclusion

Table 4 (Section 4.4) compares the greedy heuristic against the exact-for-small-components procedure on the full 1,020-record dataset. Exact-for-small-components removes marginally less mean structural weight than greedy on this corpus (0.546 vs. 0.547), which is the direction favored by the stated objective, and the paired pooled Copeland nDCG difference is effectively zero (mean $\Delta = -0.0000358$, 95% CI $[-0.000107, 0.0]$). Exact-for-small-components is exact only on strongly connected components of at most 10 nodes, falling back to greedy above that size; it is not a globally exact solver for the whole graph, and we do not describe it as one. This comparison is, by itself, sufficient to answer whether more accurate optimization of the same structural objective changes the paper’s retrieval conclusion: it does not.

7 FAILURE TAXONOMY AND DIAGNOSTIC ANALYSIS

Section 6 showed that repair’s retrieval effect is usually null and occasionally negative. This section asks why, decomposing the same 1,020-record pooled corpus into six deterministic, mutually exclusive failure classes (Table 10; Figure 6).

Table 10. Rule-based failure-class taxonomy, 1,020 query×regime records.

Class	Count	%	Mean Δ nDCG
Repair-inactive	652	63.9	0.000
Tail-only change	210	20.6	0.000
Wrong-direction repair	55	5.4	−0.034
Metric-neutral change	54	5.3	0.000
Extraction-insensitivity	26	2.5	+0.014
Unknown / mixed	23	2.3	+0.036

7.1 Rule-Based Taxonomy Assignment

Failure labels are assigned by a fixed decision sequence over the repaired and unrepaired graph/ranking pair for each query×regime record, not by a free-form manual coding pass. The rule set first checks whether repair changes graph topology at all (**repair-inactive**); then whether the extracted ranking is unchanged despite a graph change (**extraction-insensitivity**); then whether any ranking change is confined below the evaluation cutoff while Δ nDCG remains zero (**tail-only change**); then whether the top- k ranking changes but nDCG remains zero because relevance grades are unchanged (**metric-neutral change**); then whether Δ nDCG is negative (**wrong-direction repair**); and assigns the residual cases to **unknown/mixed**. The class definitions were fixed once in code and applied uniformly to all 1,020 records. This makes the taxonomy exactly reproducible, but it also means there is no separate adjudication stage or inter-annotator agreement statistic; the taxonomy is limited to mechanisms expressible in the engineered rules.

Repair-inactive (63.9%, the dominant class) covers cases where FAS repair removes negligible or zero edge weight, so the repaired and unrepaired rankings coincide by construction — the direct query-level manifestation of the near-acyclic regimes in Section 5. **Tail-only change** (20.6%) covers cases where repair does alter the underlying graph ranking, but only below the evaluation cutoff, leaving $\text{nDCG}@k$ unchanged: the structural intervention is real but invisible to the retrieval metric at the depth we score. **Metric-neutral change** (5.3%) covers cases where repair changes which documents occupy the top ranks without changing their relevance grades, so nDCG is unaffected even though the ranking itself differs. **Extraction-insensitivity** (2.5%) covers cases where the Copeland or balance scoring rule (Section 3.5) is itself insensitive to the specific edges repair removes, so the score computed from the repaired graph reproduces the unrepaired ranking. Together, these four classes — 91.7% of all records — describe mechanisms by which a structural intervention fails to reach the retrieval metric at all, not mechanisms by which repair actively harms retrieval.

Wrong-direction repair (5.4%) is the only class with a materially negative mean Δ nDCG (−0.034): here repair changes the ranking and the change moves relevant documents down, not up. This is the class in which repair is actively harmful, and it is a minority of cases even within the already-small set of records where repair is retrieval-active at all. **Unknown/mixed** (2.3%) covers records that do not cleanly fit the other five rule-defined categories. The counterfactual analysis underlying this taxonomy additionally finds that *no repair* is the minimal intervention that would have avoided every wrong-direction case — i.e., for the harmful subset, keeping the unrepaired ranking would have been sufficient, and no repaired variant recovers the loss.

Two mechanisms recur across these classes and connect back to earlier sections. First, candidate ordering and graph construction jointly determine whether repair has anything to act on: Section 5 showed this at the graph level (cyclicity

Table 11. Additional negative diagnostic evidence beyond the main repaired-versus-unrepaired comparison.

Analysis	Protocol	Quantitative summary	Interpretation
Regret decomposition	1,020 pooled records; separate counterfactual comparisons against the current repair-based method	Mean foregone nDCG: repair choice 0.0019 (3.7% of mean total regret), extraction 0.0041 (8.1%), fusion 0.0085 (16.6%), selector policy 0.0106 (20.7%), graph construction 0.0390 (76.6%), missing candidate information 0.0464 (91.1%)	Repair choice is a minor contributor relative to graph construction and candidate-set limits; percentages are non-additive counterfactual shares, not a causal partition.
Learned selector	Features {BEW, disagreement, SCC count, cyclicity}; random 60/20/20 split plus leave-one-dataset-out	Best test policy nDCG is a fixed disagreement threshold (0.5360) versus never repair 0.5279; learned logistic 0.5233 and tree 0.5212 are lower	No learned selector improves over the best simple heuristic overall.
Failure-pattern prediction	Grouped 60/20/20 split by independent query; logistic, shallow tree, random forest	Best operational test metrics are ROC-AUC up to 0.885 and PR-AUC up to 0.325 on rare help/harm or any-non-neutral targets; no permutation-validated deployment rule is retained in the reports	Moderate separability does not translate into a validated operational selector under severe class imbalance.
Counterfactual stress test	40 generated counterfactuals, 34 accepted for review	For the main Copeland hybrid: 27 qrel-valid neutral cases, 0 valid help cases, 0 valid harm cases, 7 invalid by qrel-reuse checks	Synthetic counterfactual generation did not produce scientifically usable non-neutral cases in this study.

by regime), and the repair-inactive class shows it at the query level. Second, fusion can suppress a graph-level change from ever reaching the final ranking; for the Copeland component under the reciprocal-rank-fusion combination used in the main hybrid ($\alpha = 0.3$), a repair-induced change to the raw graph ranking is masked by fusion — the fused ranking is unchanged even though the underlying graph ranking is not — in 14.7% of query-method comparisons where the graph ranking changed at all in a supporting extraction/fusion audit; the corresponding rate varies by component and fusion mode, from 4.8% to 26.4%. This is a contributing mechanism for a subset of the null results above, not a single explanation for all of them: it is consistent with, and partially overlaps, the tail-only-change and extraction-insensitivity classes, but the taxonomy’s six categories should be read as the more complete account.

These auxiliary diagnostics sharpen the same conclusion from different angles. The regret audit shows that, on this pooled corpus, most mean foregone nDCG is associated with graph construction and candidate-set limitations rather than with the repair decision itself. The learned-selector study reaches the same endpoint operationally: simple disagreement-threshold heuristics match or beat the learned policies, and no learned selector outperforms the best fixed rule overall. The failure-pattern predictors provide some class separation in ROC space, but precision-recall remains modest on the rare positive cases that would matter for deployment. Finally, the counterfactual-generation stress test does not rescue sample size: after acceptance and qrel-reuse validity checks, it yields no scientifically valid non-neutral case for the main method. We therefore use these studies as negative diagnostic evidence, not as the basis for a deployable repair-trigger policy.

The practical interpretation is direct: for a retrieval pipeline considering whether to add graph repair, the taxonomy indicates that the typical outcome is inactivity or an invisible tail-only change, a small fraction of cases are neutral or actively harmful, and a positive mean whose interval does not extend below zero appears in only one dataset/regime

cell (Section 6). This is a diagnostic account of *why* repair behaves this way in the cases observed, not a validated predictive rule for deciding in advance whether a new query or dataset will fall into one class or another; we return to this distinction in Section 11.

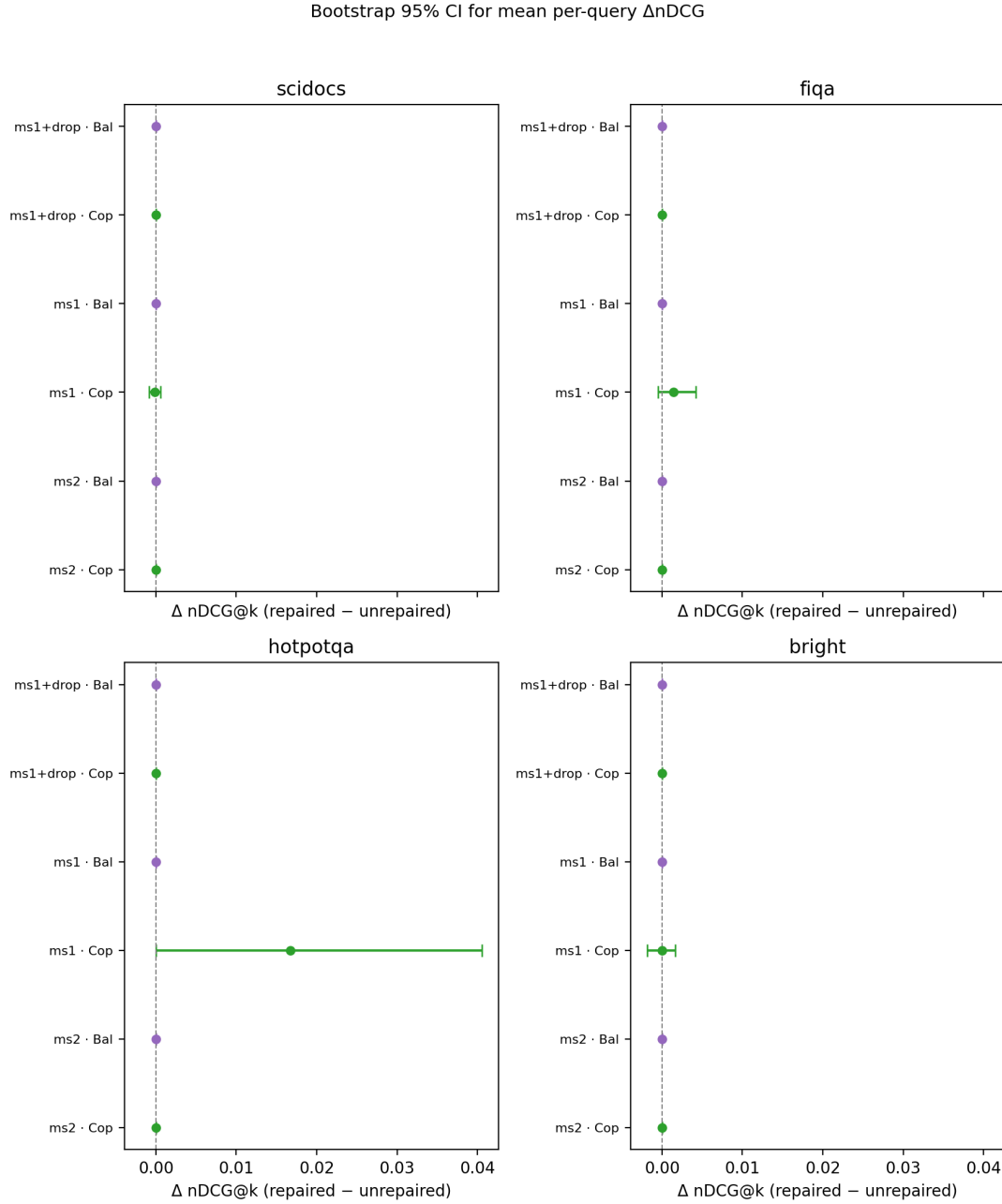


Fig. 4. Bootstrap mean Δ nDCG@k (repaired – unrepaired hybrid ranking) with 95% confidence intervals, for Copeland and balance hybrids, across four datasets and three vote-extraction regimes (24 cells total, six per dataset panel). Twenty cells are degenerate at $[0, 0]$; among the four active ms1/Copeland cells, only HotpotQA's interval does not extend below zero.

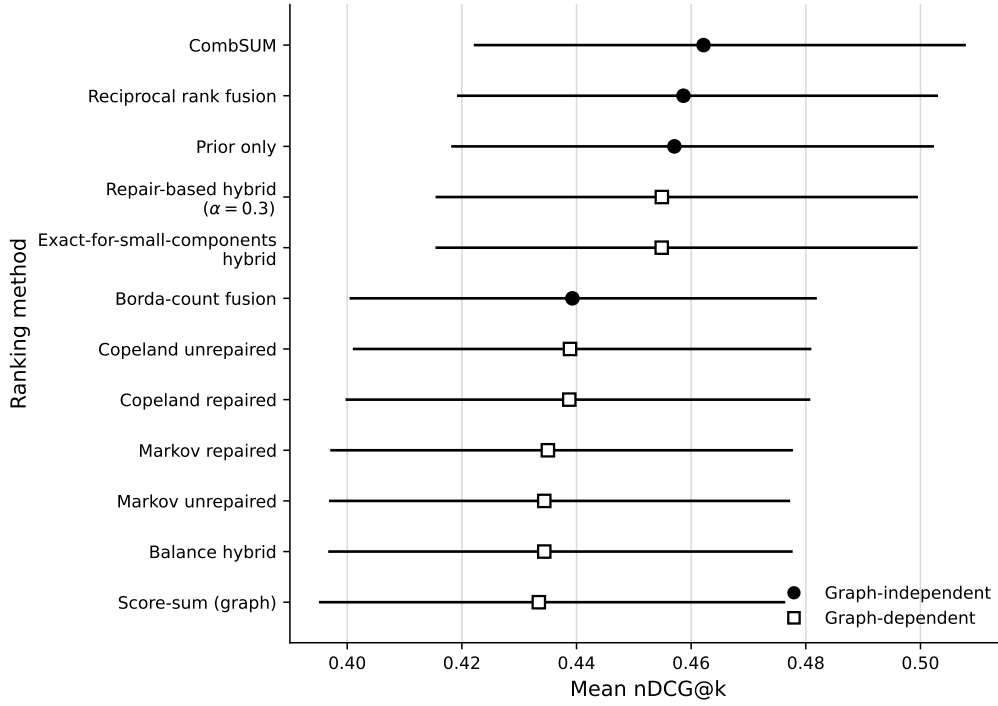


Fig. 5. Pooled mean nDCG@ k by ranking method across 1,020 query×regime records from the failure-mining evaluation corpus. Points show method means; horizontal whiskers show 95% clustered-bootstrap confidence intervals computed over 342 independent query clusters. Methods are ordered by point estimate and grouped by whether their score depends on the preference graph. CombSUM and reciprocal rank fusion have the two highest pooled mean nDCG point estimates, followed by the prior ranking; all graph-dependent methods are lower by point estimate. The intervals shown are marginal method-wise confidence intervals, not paired tests between methods.

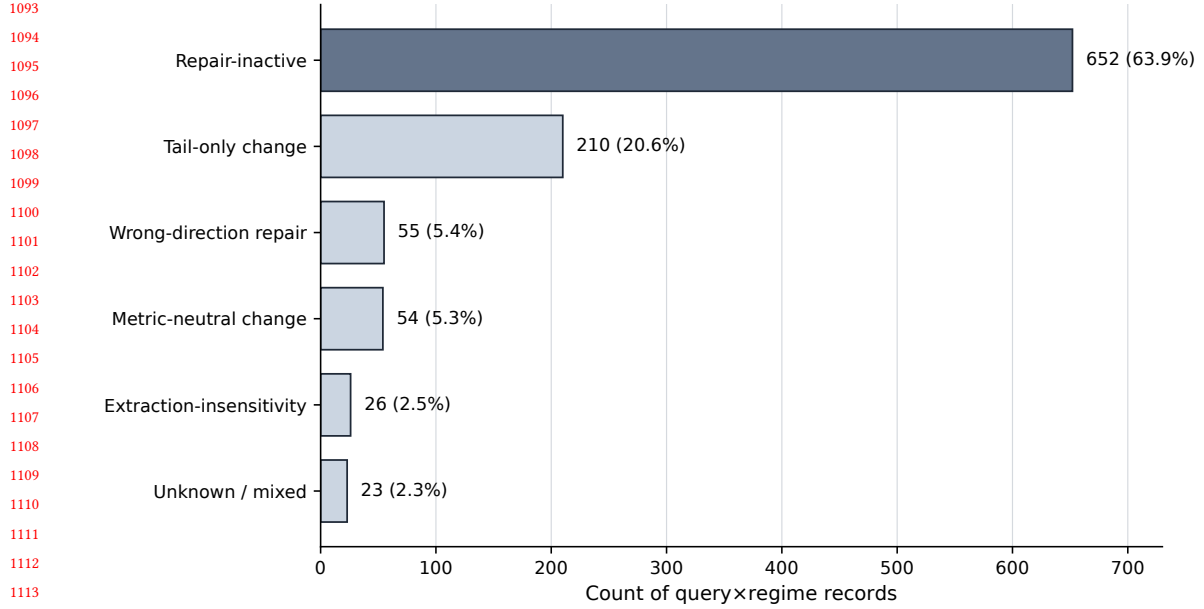


Fig. 6. Distribution of rule-based failure classes across 1,020 query×regime records. Repair-inactive (63.9%) and tail-only change (20.6%) account for the large majority of cases and describe mechanisms by which a structural repair fails to reach the retrieval metric at all, rather than mechanisms of active harm. Wrong-direction repair (5.4%) is the only class with a materially negative mean $\Delta nDCG$ and represents the minority of cases in which repair is actively unhelpful. See Table 10 for exact counts, percentages, and mean $\Delta nDCG$ per class.

8 BOUNDED REAL-LLM VALIDATION

The main evidence base (Sections 5–7) uses mechanical, score-derived votes from BM25, TF-IDF, and MiniLM (Section 4.2). This section reports a bounded check of whether the same regime-conditional pattern persists under genuine large-language-model pairwise judgments. The paired summary in Table 12 uses OpenAI pairwise judgments on SciDocs, HotpotQA, and FiQA. Separately, we analyze a protocol-distinct multi-provider failure-mining corpus with Cohere/Azure pairwise judgments on FiQA, HotpotQA, and BRIGHT; because it compares repaired and unrepaired Markov rankings under a different query sample and sampling design, we do not pool it with Table 12 or treat it as a direct replication.

Cyclicity under real LLM pairwise judgments is, as under the mechanical votes, regime- and dataset-sensitive: SciDocs and HotpotQA form a high-cyclicity setting (92.0% and 80.0% of queries), while FiQA is low-cyclicity in this run (10.0%). Repaired-versus-unrepaired Copeland Δ nDCG is small throughout: SciDocs shows a small *negative* effect with a 95% CI that excludes zero (-0.0010 , $[-0.0019, -0.0002]$); HotpotQA and FiQA are both degenerate at exactly zero. We report the SciDocs result plainly rather than softening it: the one real-LLM cell with a confidently non-null effect in this study is negative, not positive, reinforcing rather than contradicting the decoupling pattern of Section 6.

The protocol-distinct multi-provider corpus is directionally consistent with the same picture while broadening scope to BRIGHT and a second provider family. Across its 200 query-regime records, repair is inactive in all 62 ms2 records and all 69 ms1_drop_mutual records; in ms1, it yields one help case and one harm case among 69 records. Because that corpus compares repaired and unrepaired Markov rankings rather than the Copeland-hybrid pair summarized in Table 12, we use it only as corroborative scope evidence rather than as a pooled estimate.

This evidence is corroborative, not primary. Query budgets are small (10–50 per dataset, an order of magnitude or more below the mechanical-vote per-regime counts in Table 2), the paired cross-dataset summary itself uses a single provider, and FiQA’s count reflects a usable-query-constrained subset of a larger target without a recorded exclusion audit. The separate multi-provider corpus extends scope but follows a different protocol and method pair. We treat this section as a bounded check that the structural/retrieval decoupling pattern is not an artifact specific to mechanical score-derived votes, not as an independent confirmation at the same evidentiary standard as Sections 5–7, and we do not draw conclusions from it beyond that limited scope.

9 EFFICIENCY AND PRACTICAL CONSIDERATIONS

We summarize existing runtime and memory evidence from the real-data pipeline (Section 4.6); this section reports only measurements already collected, not a new benchmarking exercise. On the full evaluation dataset, greedy repair’s measured wall-clock cost is effectively at the resolution floor of our timing instrumentation (≈ 0 s per query in the pooled aggregation); exact-for-small-components adds a modest, measurable overhead (mean 0.062s per query, mean peak RSS $\approx 1,646$ MB) relative to the same baseline. Given the retrieval-quality comparison in Section 6, this additional cost is not accompanied by a meaningful change in pooled Copeland nDCG on this corpus: the paired difference is effectively zero within the reported interval. We therefore report the efficiency evidence descriptively rather than as a general recommendation about the favorable operating point for all deployments.

We are explicit about the limits of this evidence. Runtime and memory were measured on a single, unbenchmarked local development machine (Section 4.6), not a controlled hardware configuration, so absolute figures should not be read as generalizing to other environments; memory is a coarse before/after resident-set-size snapshot, not instrumented peak-memory tracking. We are not aware of a publicly documented, comparable real-data memory benchmark

Table 12. Real-LLM pairwise pilot summary. Δ nDCG is repaired minus unrepaired Copeland hybrid.

Dataset	Queries	Cyclic%	Δ nDCG (95% CI)
SciDocs	50	92.0	$-0.0010 [-0.0019, -0.0002]$
HotpotQA	20	80.0	0.0 [0, 0]
FiQA	10 ^a	10.0	0.0 [0, 0]

^a 10 of a 20-query target contribute usable paired records in the stored summary; the logs record no API errors for this run, but no per-query exclusion audit is available, so selection bias remains possible.

Table 13. Planned CARB release statistics.

Statistic	Value
Independent queries	440
Query×regime records	1,020
Method-output fields in wide record	366
Feature groups	14+
Source datasets	SciDocs, FiQA, HotpotQA, BRIGHT

beyond the single comparison reported here, and we do not extrapolate from it to a general efficiency claim. A separate, purely synthetic runtime study elsewhere in the supporting materials (dense synthetic graphs, up to 100 items) shows repair cost growing steeply with graph size, but that study uses artificially dense graphs unlike the sparser real preference graphs evaluated here, and we do not treat it as evidence about the real pipeline’s scaling behavior.

10 CARB BENCHMARK

A planned companion resource, which we refer to as the *Consistency-Aware Reranking Benchmark* (CARB), would package the derived graph-theoretic features, repair outcomes, ranking scores, and diagnostic labels underlying Sections 5–7 for follow-on preference-graph data-quality research. In the current revision, however, CARB is not yet packaged as anonymous supplementary material, so we do not present it as a released contribution.

The richest stored layout is a wide query×regime record. Table 13 therefore summarizes the planned scope as 440 independent queries across the four datasets studied here and 1,020 query×regime records (Section 3.2). The reported figure “366” refers to the number of method-output fields stored in that wide record layout, not to 366 independent methods attached to every long-form observation used elsewhere in the paper. Those fields are organized into 14 or more feature groups spanning pre-repair graph structure (cyclicity, SCC size, edge counts), repair outcomes (FAS weight removed), ranking scores, and the failure-class labels of Section 7.

Provenance is derived entirely from the four public source datasets already cited (Section 4.1) and from the mechanical ranker scores and repair pipeline described in Sections 3 and 4; no raw document text or third-party corpus content is intended for redistribution, and any eventual redistribution would be feature-only (graph statistics, scores, and labels), respecting the licensing terms of the underlying source datasets. If prepared for release, a data card should document schema, provenance, known limitations (including the BEW/PIC circularity noted in Section 5 and the rule-based nature of the failure taxonomy), and intended uses. No empirical claim in this paper depends on CARB’s public availability.

11 DISCUSSION

The results assembled in Sections 5–9 support a single, consistent interpretation: structural consistency and downstream retrieval utility are related but separable dimensions of data quality, and improving the former is not a reliable route to improving the latter. We discuss what this decoupling means, why it arises mechanically rather than incidentally, and what would be required to move beyond a diagnostic account toward a predictive one.

Why repair can succeed mathematically but fail operationally. FAS repair (Eq. (4)) is defined entirely in terms of the preference graph’s own edge weights; nothing in that objective references the relevance judgments a retrieval metric is scored against. Section 5 confirms repair does what it is designed to do — reduce backward-edge weight and pairwise inconsistency relative to a reference ranking — exactly when the graph is cyclic. But because the objective and the downstream metric are distinct, a repair step can succeed completely on its own terms while being irrelevant, neutral, or actively unhelpful once fused into a final ranking. The failure taxonomy (Section 7) gives this abstract observation concrete mechanisms: inactivity when the graph has little to repair, invisibility when the change falls below the evaluation cutoff, insensitivity when the scoring rule does not register the specific edges removed, and, in a minority of cases, active harm when the removed edges happened to be aligned with relevance rather than against it.

Construction and extraction, not repair alone, govern the outcome. A recurring theme across Sections 5 and 6 is that vote-extraction regime, not the repair step, is the dominant determinant of whether repair has any work to do at all (Table 6), and that the choice of extraction rule (Copeland vs. balance, Eqs. (5) and (6)) shapes whether a structural change reaches the final score once it does occur — the balance hybrid is uniformly repair-invariant across every cell we test (Table 8), while Copeland is not. Preference construction, graph repair, and ranking extraction are three separable design choices with distinct empirical consequences, and treating “graph repair” as a single monolithic intervention obscures where its effects actually originate.

Most foregone retrieval quality lies outside the repair choice. A complementary regret audit on the same 1,020-record pooled corpus reinforces this ordering of influences. Relative to the current repair-based method, separate counterfactual comparisons attribute mean foregone nDCG of 0.0390 to graph-construction choice (76.6% of mean total regret) and 0.0464 to missing candidate information (91.1%), compared with only 0.0019 for the repair choice itself (3.7%). Extraction choice (0.0041), fusion choice (0.0085), and selector policy (0.0106) lie in between. These percentages are not additive and should not be read as a causal decomposition; each is defined against a different counterfactual baseline. They nonetheless make clear that repair choice is a comparatively small part of the total performance gap on this corpus.

Fusion is both a stabilizer and a suppressor. The hybrid combination (Eq. (7)) mixes a graph-derived component into a prior ranking at a fixed weight α . This stabilizes the final ranking against noisy or spurious graph-level changes — a virtue when the graph signal is unreliable — but the same mechanism can suppress a genuine, repair-induced structural change before it reaches the retrieval metric (Section 7, 14.7% suppression rate for the main hybrid’s component/fusion combination). Which effect dominates is not knowable from α alone; it depends on the interaction between how much the graph changed and how the fusion weight discounts that change relative to the prior, which is why we treat fusion suppression as a diagnostic finding rather than a tunable knob to optimize in this study.

Dataset-specific effects resist a simple structural explanation. HotpotQA is the one dataset with a positive mean whose interval does not extend below zero, and it is also the least cyclic of the four datasets under ms1 (Table 6) — the opposite of what a naive “more inconsistency, more room for repair to help” account would predict. We do not have a confirmed explanation for why HotpotQA behaves this way; plausible candidates include its multi-hop

structure interacting differently with the three constituent rankers, or dataset-specific relevance grading conventions, but distinguishing between these would require new experiments outside this paper’s scope. We flag this as the clearest open question this study raises rather than resolves.

Implications for preference-graph ranking research. Taken together, these findings suggest that future work in this area should report structural and retrieval outcomes separately by default, rather than assuming the former predicts the latter, and should treat vote construction as a first-class experimental factor rather than fixed preprocessing. Stronger optimization of the structural objective alone is not sufficient to change this picture: Section 6’s exact-for-small-components comparison shows that solving the same objective more thoroughly than a greedy heuristic does not translate into a retrieval-quality gain, because the objective being optimized more thoroughly is still not the metric ultimately being evaluated. A genuinely predictive criterion for *when* repair will help — as opposed to the retrospective taxonomy offered here — would need to identify, in advance of running repair, some observable property of a query’s graph or candidate set that correlates with which failure class it will fall into; our exploratory attempts in this direction do not produce such a rule. Learned logistic and shallow-tree selectors do not beat a simple disagreement-threshold policy overall, and failure-pattern classifiers reach ROC-AUC in the mid-0.8s but only modest precision-recall on these rare positive cases. We therefore do not claim a validated predictive criterion.

Answering the original question. We set out to ask when repairing preference-graph inconsistency improves downstream retrieval quality. The answer supported by this study is: rarely, and only when the vote-extraction regime has already made the graph substantially cyclic, and even then in a dataset-dependent way that is not explained by cyclicity severity alone. The more actionable finding is the negative one — knowing when repair is inactive, invisible, or neutral is itself useful guidance for a practitioner deciding whether to add a repair step to a retrieval pipeline, since it identifies the large majority of cases in which the intervention would not change the retrieval metric.

12 LIMITATIONS

We state the limitations of this study candidly, several of which have already been flagged at first relevance earlier in the paper rather than held back for this section alone.

Real-LLM scale. Section 8’s paired cross-dataset pilot uses 10–50 queries per dataset and a single provider. We also analyze a separate 200-record multi-provider failure-mining corpus with Cohere/Azure judgments and BRIGHT coverage, but that corpus uses a different method pair, query sample, and protocol. Together these streams are sufficient to show that the main decoupling pattern is not obviously confined to one provider or to the mechanical-vote setting, but not sufficient to establish how it behaves under LLM judgments in general.

No validated predictive selector. The failure taxonomy (Section 7) explains, retrospectively, why repair succeeded or failed in the cases observed. It is not a validated criterion for predicting, in advance, whether repair will help on a new query or dataset; exploratory learned selectors do not beat a fixed disagreement threshold overall, and failure-pattern models show limited precision-recall on rare positive cases (Section 11). We do not claim otherwise.

Rule-based rather than human-annotated taxonomy. The failure classes of Section 7 are assigned by a pre-specified rule set over graph changes, ranking changes, and observed $\Delta nDCG$, not by an independent human annotation exercise. This makes the taxonomy reproducible, but it also means the analysis can only surface mechanisms expressible in those rules and provides no inter-annotator agreement statistic.

Protocol differences across experiment families. The vote-extraction suite (Sections 5–6, first two subsections) and the pooled failure-mining corpus (Section 6’s baseline comparison, Section 7) share method implementations but

not query populations; we have flagged this distinction at each point it matters (Section 4.3), but a reader assembling numbers across sections should not treat the two corpora as interchangeable.

HotpotQA sample size. HotpotQA’s ms1 cell — the one regime with a positive mean whose interval does not extend below zero — has $n = 52$ queries, smaller than SciDocs or FiQA’s corresponding cells. The bootstrap CI we report (Table 8) already reflects this sample size, but a result resting on 52 queries warrants caution before generalizing beyond this study.

Structural metrics tied to the repair objective. BEW and PIC (Section 5) are computed against a reference ranking derived from the same relevance judgments used for downstream nDCG evaluation. This is a qrels-anchored structural diagnostic, not an independent measure of graph correctness, and should not be read as external validation that repair improves some ground truth beyond the evaluation data itself.

Incomplete memory benchmarking. As stated in Section 9, memory measurements are a coarse before/after snapshot on a single, unbenchmarked machine, and we are not aware of a documented real-data memory benchmark beyond the one comparison reported. We do not make a general efficiency claim beyond that bounded evidence.

Fixed hybrid weight. The main hybrid uses $\alpha = 0.3$, the fixed setting present in the stored outputs. We did not identify a held-out validation study establishing this value as optimal for the four-dataset protocol, so the conclusions about fusion suppression should not be read as a sensitivity analysis over α .

FiQA usable-query constraint. In the OpenAI pairwise pilot, FiQA contributes 10 usable paired records from a 20-query target. The stored logs record no API errors, but they do not include a per-query exclusion audit, so we cannot verify whether the completed subset is representative of the full target. Selection bias is therefore possible in that FiQA real-LLM cell.

Derived-data licensing constraints. The planned CARB resource (Section 10) is restricted to feature-only redistribution, respecting the source datasets’ own licensing terms; it is not a redistribution of raw document text, and any use of CARB remains subject to the original datasets’ licenses for the underlying corpora.

No anonymous CARB package in this draft. Although Section 10 describes the intended scope of CARB, this revision does not yet include a review-ready anonymous package for that resource. We therefore do not treat CARB’s availability as part of the paper’s evidentiary contribution.

No claim of universal external validity. The central findings summarized in Section 11 are established on four datasets, three vote-extraction regimes, and a three-ranker mechanical voting scheme (BM25, TF-IDF, MiniLM). We do not claim these findings generalize to arbitrarily different ranker families, domains, or preference-construction procedures beyond what Section 8’s bounded check already covers.

13 CONCLUSION

This paper studied preference-graph inconsistency as a measurable structural data-quality dimension and evaluated feedback-arc-set repair as a candidate improvement technique, across four public retrieval benchmarks and three vote-extraction regimes. Repair reliably improves structural consistency (reduced backward-edge weight and pairwise inconsistency) whenever the underlying graph is substantially cyclic, and vote-extraction regime, not repair itself, is the dominant determinant of whether that condition holds at all. That structural improvement, however, does not reliably translate into a downstream retrieval-quality gain: twenty of twenty-four dataset-regime-method cells show no retrieval effect whatsoever, and among the four cells where the graph is substantially cyclic, only one — HotpotQA under the most permissive vote-extraction regime — has a positive mean with interval $[0, 0.0405]$, and that bounded result is not explained by cyclicity severity alone. Strong, simple aggregation baselines that never inspect graph structure

at all, CombsUM and reciprocal rank fusion in particular, remain competitive with or superior to every graph-based method evaluated, including repaired hybrids. A six-class rule-based failure taxonomy indicates that the typical outcome of applying repair is inactivity or an evaluation- invisible tail change, that active harm is a small minority of cases, and that non-negative retrieval effects are rare and regime-specific. These findings were corroborated, within a bounded scope, by an OpenAI pairwise pilot on SciDocs, HotpotQA, and FiQA and by a separate protocol-distinct Cohere/Azure failure-mining corpus that includes BRIGHT. The practical takeaway is deliberately modest rather than promotional: a practitioner deciding whether to add graph repair to a retrieval pipeline should expect it to succeed as a structural intervention far more often than as a retrieval-quality one, and should budget for that distinction rather than assume the two travel together.

14 DATA AVAILABILITY AND REPRODUCIBILITY

Data availability. All four source datasets (SciDocs, FiQA, HotpotQA, BRIGHT) are publicly available from their original sources [3, 12, 15, 19] under their respective licenses; this paper introduces no new raw-text corpus. The derived preference-graph features, repair outcomes, and diagnostic labels underlying Sections 5–7 motivate the planned CARB companion resource (Section 10), feature-only and license-compatible with the source datasets; that resource is not yet packaged as anonymous supplementary material in this draft.

Artifact availability. The experiments are implemented in code and stored outputs maintained by the authors, but no public URL is provided in this anonymous manuscript. A citable code release is intended for the camera-ready stage rather than claimed as part of the present anonymous draft.

Reproducibility. Every table in Sections 5–6 is generated from stored intermediate outputs by scripted, deterministic aggregation; the vote-extraction protocol (Section 3.1), regime definitions (Section 3.2), and bootstrap procedure (Section 4.5) are fully specified with fixed parameters (thresholds, α , bootstrap seeds and resample counts) so that the mechanical-vote tables and figures can be regenerated from the same stored inputs without rerunning large retrieval or graph-construction jobs. The real-LLM summaries likewise depend on stored API outputs, but a full rerun of those experiments would require commercial API access and may not be exactly reproducible if provider-side models change over time.

14 REFERENCES

- [1] Nir Ailon, Moses Charikar, and Alantha Newman. 2008. Aggregating Inconsistent Information: Ranking and Clustering. *J. ACM* 55, 5 (2008), 23:1–23:27. <https://doi.org/10.1145/1411509.1411513>
- [2] Christopher Burges, Tal Shaked, Erin Renshaw, Ari Lazier, Matt Deeds, Nicole Hamilton, and Greg Hullender. 2005. Learning to Rank Using Gradient Descent. In *Proceedings of the 22nd International Conference on Machine Learning*. ACM, 89–96. <https://doi.org/10.1145/1102351.1102363>
- [3] Arman Cohan, Sergey Feldman, Iz Beltagy, Doug Downey, and Daniel Weld. 2020. SPECTER: Document-level Representation Learning using Citation-informed Transformers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, Online, 2270–2282. <https://doi.org/10.18653/v1/2020.acl-main.207>
- [4] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. 2009. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, New York, NY, USA, 758–759. <https://doi.org/10.1145/1571941.1572114>
- [5] Cynthia Dwork, Ravi Kumar, Moni Naor, and Dandapani Sivakumar. 2001. Rank Aggregation Methods for the Web. In *Proceedings of the 10th International Conference on World Wide Web*. ACM, Hong Kong, Hong Kong, 613–622. <https://doi.org/10.1145/371920.372165>
- [6] Ronald Fagin, Ravi Kumar, and Dandapani Sivakumar. 2003. Comparing Top k Lists. *SIAM Journal on Discrete Mathematics* 17, 1 (2003), 134–160. <https://doi.org/10.1137/S0895480102412856>
- [7] Edward A. Fox and Joseph A. Shaw. 1994. Combination of Multiple Searches. In *The Second Text REtrieval Conference (TREC-2) (NIST Special Publication, 500-215)*, Donna K. Harman (Ed.). National Institute of Standards and Technology, Gaithersburg, MD, USA, 243–252. <https://trec.nist.gov/pubs/trec2/papers/txt/23.txt>

- [8] Xiang Jiang, Lek-Heng Lim, Yuan Yao, and Yinyu Ye. 2011. Statistical Ranking and Combinatorial Hodge Theory. *Mathematical Programming* 127, 1 (2011), 203–244. <https://doi.org/10.1007/s10107-010-0419-x>
- [9] Claire Kenyon-Mathieu and Warren Schudy. 2007. How to Rank with Few Errors. In *Proceedings of the 39th Annual ACM Symposium on Theory of Computing*. ACM, San Diego, California, USA, 95–103. <https://doi.org/10.1145/1250790.1250806>
- [10] Tie-Yan Liu. 2011. *Learning to Rank for Information Retrieval*. Springer, Berlin, Heidelberg.
- [11] Stuart E. Madnick, Richard Y. Wang, Yang W. Lee, and Hongwei Zhu. 2009. Overview and Framework for Data and Information Quality Research. *Journal of Data and Information Quality* 1, 1 (2009), 1:1–1:22. <https://doi.org/10.1145/1515693.1516680>
- [12] Macedo Maia, Siegfried Handschuh, André Freitas, Brian Davis, Ross McDermott, Manel Zarrouk, and Alexandra Balahur. 2018. WWW’18 Open Challenge: Financial Opinion Mining and Question Answering. In *Companion of the The Web Conference 2018 on The Web Conference 2018*. ACM, Lyon, France, 1941–1942. <https://doi.org/10.1145/3184558.3192301>
- [13] Sahand Negahban, Sewoong Oh, and Devavrat Shah. 2017. Rank Centrality: Ranking from Pairwise Comparisons. *Operations Research* 65, 1 (2017), 266–287. <https://doi.org/10.1287/opre.2016.1534>
- [14] Zhen Qin, Rolf Jagerman, Kai Hui, Honglei Zhuang, Junru Wu, Le Yan, Jiaming Shen, Tianqi Liu, Jialu Liu, Donald Metzler, Xuanhui Wang, and Michael Bendersky. 2024. Large Language Models are Effective Text Rankers with Pairwise Ranking Prompting. In *Findings of the Association for Computational Linguistics: NAACL 2024*, Kevin Duh, Helena Gomez, and Steven Bethard (Eds.). Association for Computational Linguistics, Mexico City, Mexico, 1504–1518. <https://doi.org/10.18653/v1/2024.findings-naacl.97>
- [15] Hongjin Su, Howard Yen, Mengzhou Xia, Weijia Shi, Niklas Muennighoff, Han yu Wang, Haisu Liu, Quan Shi, Zachary S. Siegel, Michael Tang, Ruoxi Sun, Jinsung Yoon, Serkan O. Arik, Danqi Chen, and Tao Yu. 2024. BRIGHT: A Realistic and Challenging Benchmark for Reasoning-Intensive Retrieval. *arXiv preprint arXiv:2407.12883* (2024). [arXiv:2407.12883 \[cs.IR\]](https://arxiv.org/abs/2407.12883)
- [16] Weiwei Sun, Lingyong Yan, Xinyu Ma, Shuaiqiang Wang, Pengjie Ren, Zhumin Chen, Dawei Yin, and Zhaochun Ren. 2023. Is ChatGPT Good at Search? Investigating Large Language Models as Re-Ranking Agents. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 14918–14937. <https://doi.org/10.18653/v1/2023.emnlp-main.923>
- [17] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. 2021. BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*.
- [18] Richard Y. Wang and Diane M. Strong. 1996. Beyond Accuracy: What Data Quality Means to Data Consumers. *Journal of Management Information Systems* 12, 4 (1996), 5–33. <https://doi.org/10.1080/07421222.1996.11518099>
- [19] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Brussels, Belgium, 2369–2380. <https://doi.org/10.18653/v1/D18-1259>